Universidad Tecnológica Nacional
Facultad Regional Córdoba
Catedra de Diseños de Sistemas de Información

Proyecto Práctico de Aplicación Integrador: Bolsín

CU 28: Registrar recepción de bolsín.

Diseño

Curso: 3k5

Número de entrega: E2

Fecha de entrega: 16/11/2026

Docentes:

- Ing. Laura Covaro
- Ing. Sol Vega
- Ing. Georgina Gonzáles
- Ing. Silvia Pereyra

Número de grupo: 8

Integrantes:

- 95763 - Bocek, Bruno - bocekbruno@gmail.com
- 401675- Diaz, Lautaro Nicolas - launikd@gmail.com
- 407594 – Diaz Nievas, Fabricio - diaznievasfabricio@gmail.com
- 400476 - Farfan, Gonzalo - farfangonza8@gmail.com
- 409247 - Lamazón, Violeta – violelamazon@gmail.com
- 409518 - Oberto, Oliver – oliveroberto6@gmail.com
- 81994 - Pino Videla, Carlos Emanuel - emanuelpino42@gmail.com

Índice

Índice 2
Rediseño de la realización del caso de uso de Análisis 3
Vista de Clases de Diseño 3
Diagrama de Clases 3
Máquina de Estados 4
Explicación 5
Descripción del problema 5
Justificación de la solución 5
Vista de Interacción 6
Diagrama de Secuencia 6

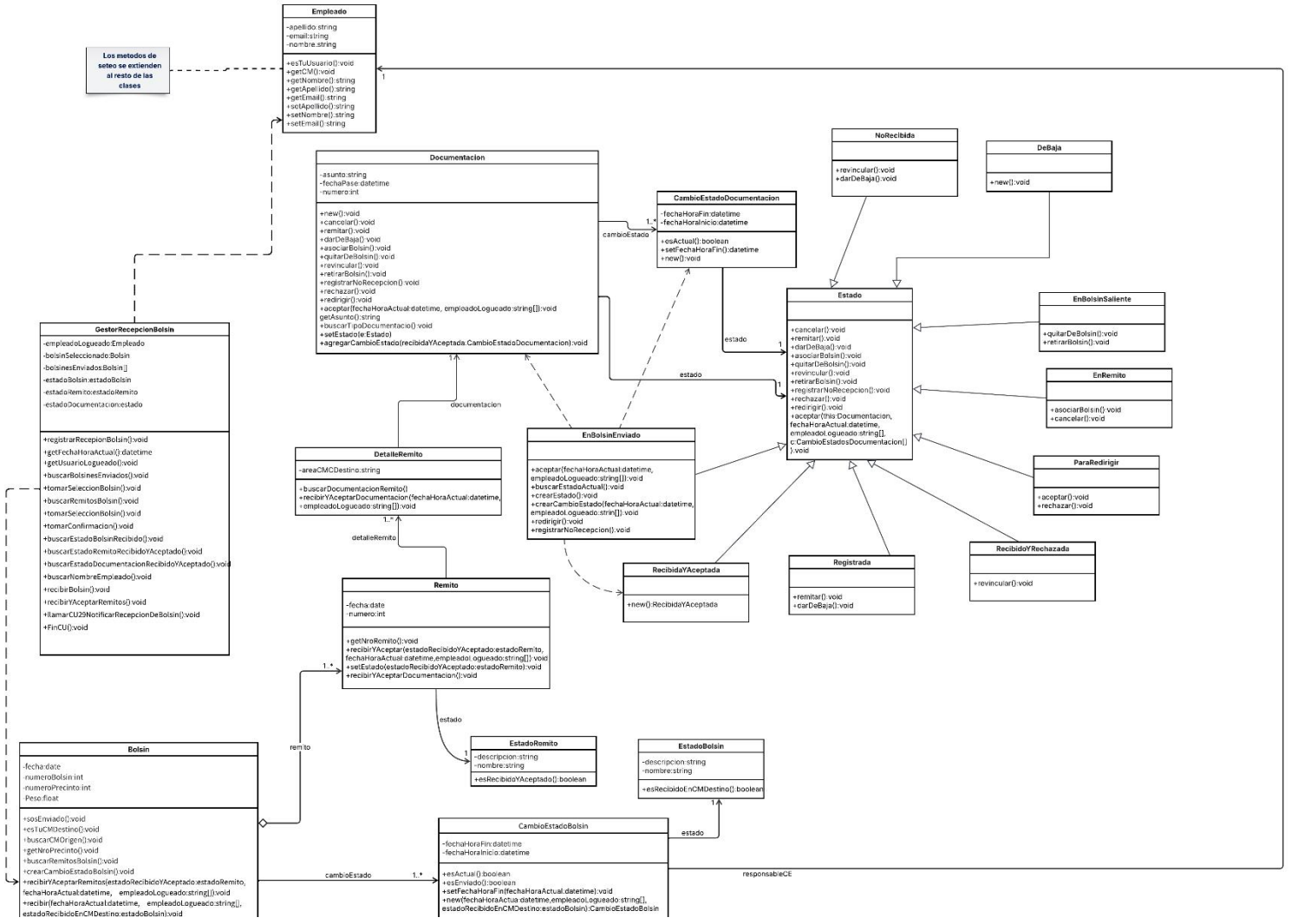
Rediseño de la realización del caso de uso de Análisis

Vista de Clases de Diseño

Diagrama de Clases

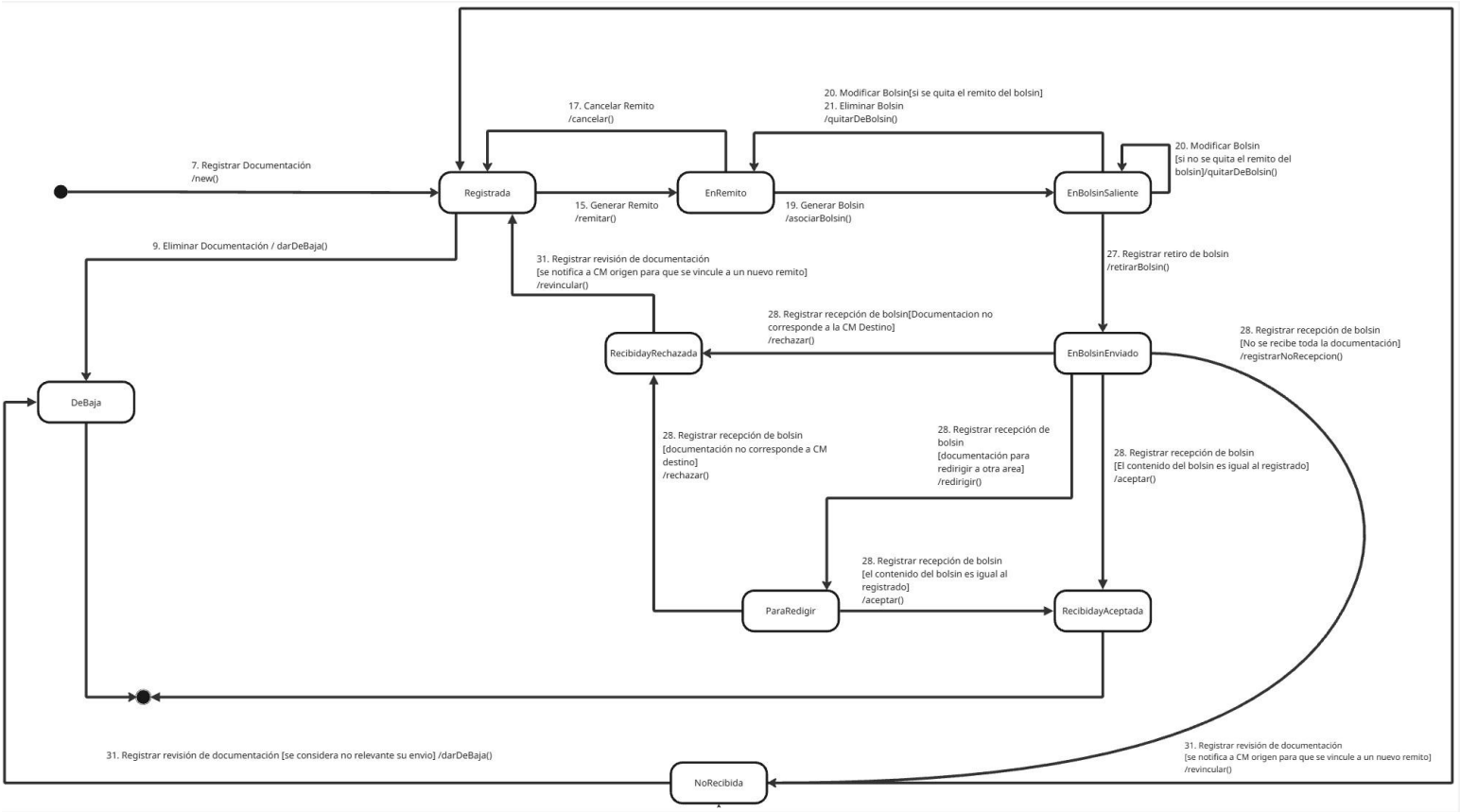
Estructura estática del rediseño del sistema para el **Caso de Uso 28 (Registrar recepción de bolsín)**, donde se especifican las clases, interfaces y relaciones necesarias para dar soporte a la resolución dinámica y polimórfica del **cambio de estado de la Documentación**.

CLAESES DE DIAGRAMA DE SECUENCIA



Máquina de Estados

En esta vista se representa la máquina de estados de la clase **Documentación**, mostrando los distintos estados por los que puede pasar y las transiciones que se producen entre ellos a lo largo de su ciclo de vida.



Explicación

Descripción del problema

La principal debilidad de gestionar los cambios de estado mediante lógica basada en estructuras condicionales (if o switch) radica en que, a medida que se incorporan nuevos estados y transiciones en el dominio los métodos de la clase de dominio tienden a convertirse en condicionales voluminosos y rígidos. Esto genera un alto acoplamiento, dificulta el mantenimiento del código e incrementa el riesgo de introducir errores involuntarios ante cualquier modificación.

Justificación de la solución

Mediante la aplicación del **patrón State**, se encapsulan los distintos estados por los que puede pasar una Documentación en objetos de estado independientes que heredan de una misma jerarquía abstracta (Estado). De esta manera, en lugar de que la clase Documentación evalúe condicionalmente en qué estado se encuentra para ejecutar una transición, delega polimórficamente dicha responsabilidad a su objeto de estado actual.
Esta solución respeta el **Principio de Abierto/Cerrado (OCP)**, permitiendo incorporar nuevos estados en el futuro sin modificar las clases existentes, y el Principio de Responsabilidad Única (SRP), al aislar la lógica de cada transición dentro de su respectiva clase de estado concreto.

Documentación - Clase concreta – Contexto: Esta clase es el contexto del patrón, cuyo comportamiento cambia según el estado en el que está. En el momento en que se le indica que debe actualizar su estado por la recepción del bolsín en la Comisión Médica destino, mediante el mensaje **aceptar (fechaHoraActual: datetime, empleadoLogueado: string []): void)** que recibe desde el DetalleRemito, delega la responsabilidad de dicho cambio de estado a su estado actual, que en términos de ejecución es EnBolsinEnviado. Recibe la fecha y hora actual y el empleado responsable para efectuar la transición.

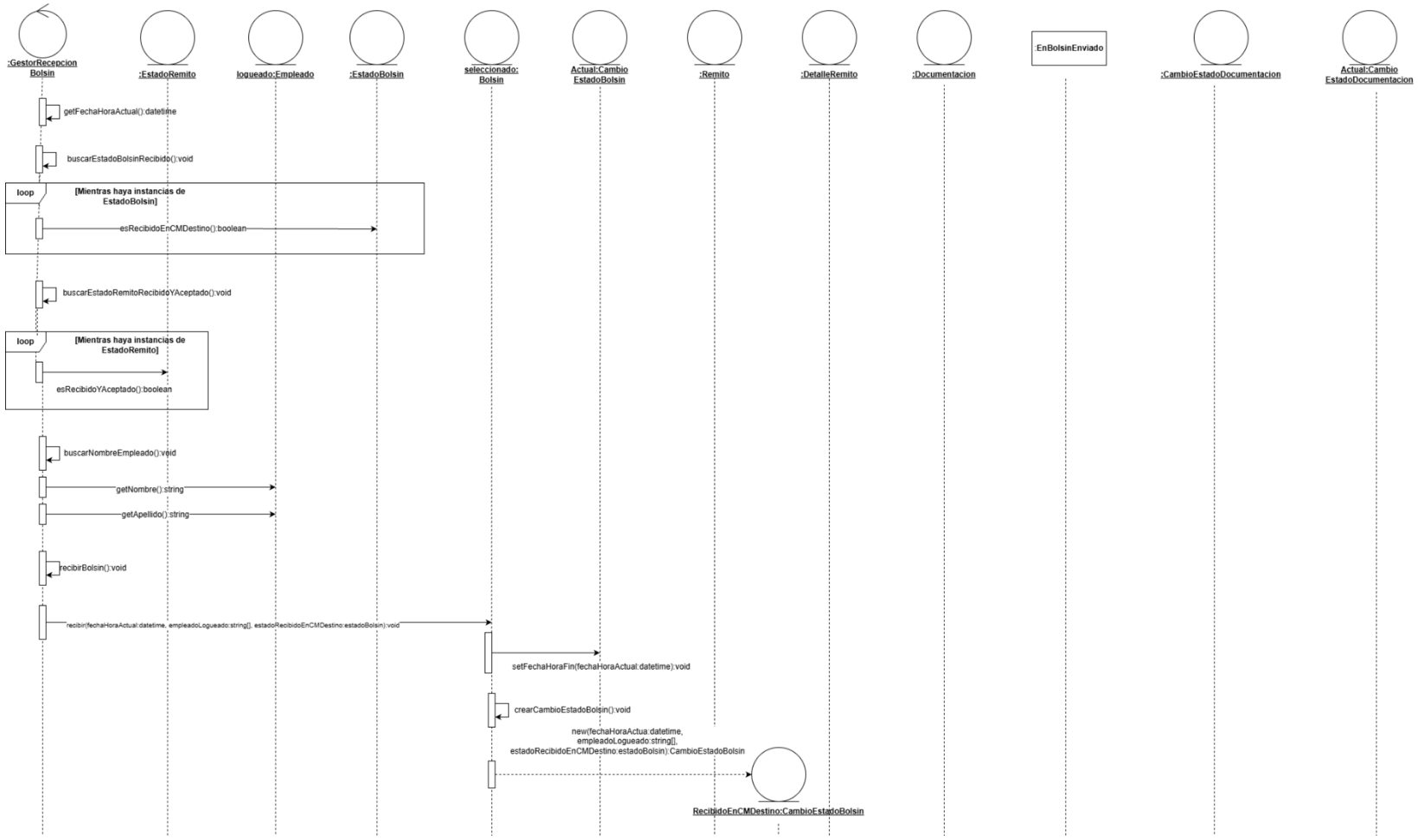
Estado - Clase abstracta: Su método aceptar(this:Documentacion, fechaHoraActual:datetime, empleadoLogueado:string[],c:CambioEstadosDocumentacion[]):void es polimórfico, porque cada una de sus clases hijas (estados concretos) le dará una implementación concreta de acuerdo con el comportamiento permitido en cada estado. Se le pasa por parámetro la instancia de la documentación (this) para indicarle el cambio realizado, la fecha/hora actual, el empleado responsable (nombre y apellido en un array) y la colección/arreglo de cambios de estado (c), permitiendo así que el estado vigente pueda finalizar el cambio de estado actual y registrar el nuevo. Se aplica en este caso una relación de herencia para el reúso de código y la definición del contrato abstracto de comportamiento.

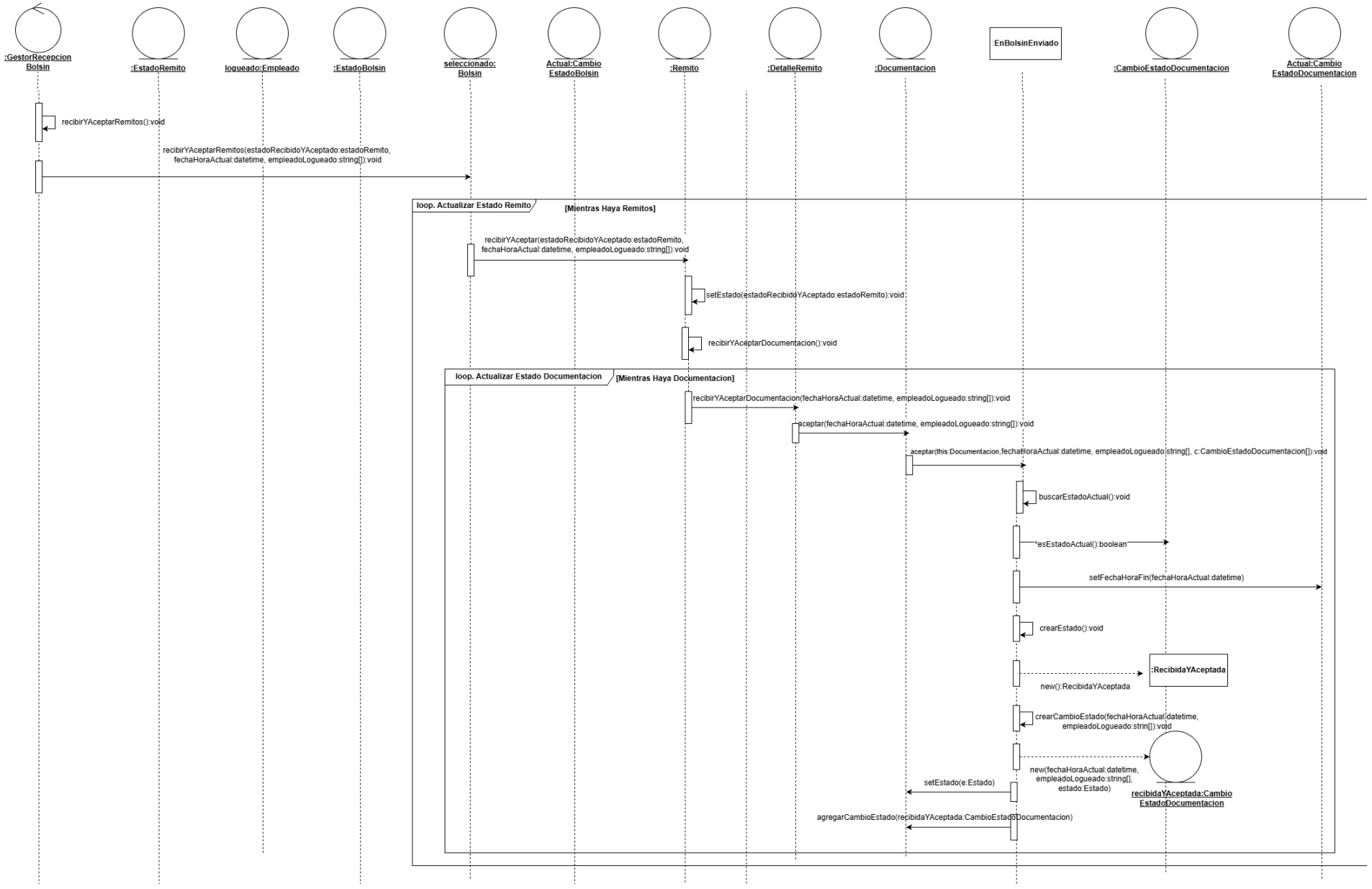
EnBolsinEnviado - Clase concreta - Estado concreto: Hereda de Estado y le da una implementación específica al método aceptar(...). Dentro del mismo, localiza el cambio de estado vigente de la documentación y le registra la fecha y hora de fin (setFechaHoraFin); luego crea/asigna el nuevo estado RecibidaYAceptada y crea la nueva instancia de CambioEstadoDocumentacion (new), enviándole por parámetro la fecha de inicio, el empleado logueado y el estado correspondiente. Por último, le indica a su objeto Documentacion que tiene un nuevo estado (setEstado) y que agregue a su arreglo/colección el nuevo cambio de estado (agregarCambioEstado).

Vista de Interacción

Diagrama de Secuencia

A continuación, se presenta el Diagrama de Secuencia correspondiente al rediseño de la realización dinámica del Caso de Uso. En esta vista se modela la transición y cambio de estado de la **Documentación** mediante la aplicación del patrón de diseño **State**, delegando las responsabilidades de transición a las clases concretas de estado.




:GestorRecepcionBolsin
:EstadoRemito
logueado:Empleado
:EstadoBolsin
seleccionado:Bolsin
Actual:CambioEstadoBolsin
:Remito
:DetalleRemito
:Documentacion
:EnBolsinEnviado
:CambioEstadoDocumentacion
Actual:CambioEstadoDocumentacion
recibirYAceptarRemitos():void
recibirYAceptarRemitos(estadoRecibidoYAceptado:estadoRemito, fechaHoraActual:datetime, empleadoLogueado:string[]):void
loop. Actualizar Estado Remito
[Mientras Haya Remitos]
recibirYAceptar(estadoRecibidoYAceptado:estadoRemito, fechaHoraActual:datetime, empleadoLogueado:string[]):void
setEstado(estadoRecibidoYAceptado:estadoRemito):void
recibirYAceptarDocumentacion():void
loop. Actualizar Estado Documentacion
[Mientras Haya Documentacion]
recibirYAceptarDocumentacion(fechaHoraActual:datetime, empleadoLogueado:string[]):void
aceptar(fechaHoraActual:datetime, empleadoLogueado:string[]):void
aceptar(this:Documentacion,fechaHoraActual:datetime, empleadoLogueado:string[], c:CambioEstadoDocumentacion[]):void
buscarEstadoActual():void
*esEstadoActual():boolean
setFechaHoraFin(fechaHoraActual:datetime)
crearEstado():void
:RecibidaYAceptada
new():RecibidaYAceptada
crearCambioEstado(fechaHoraActual:datetime, empleadoLogueado:strin[]):void
new(fechaHoraActual:datetime, empleadoLogueado:string[], estado:Estado)
recibidaYAceptada:CambioEstadoDocumentacion
setEstado(e:Estado)
agregarCambioEstado(recibidaYAceptada:CambioEstadoDocumentacion)